



Algoritmos de planificación de procesos

Sistemas Operativos 1151018

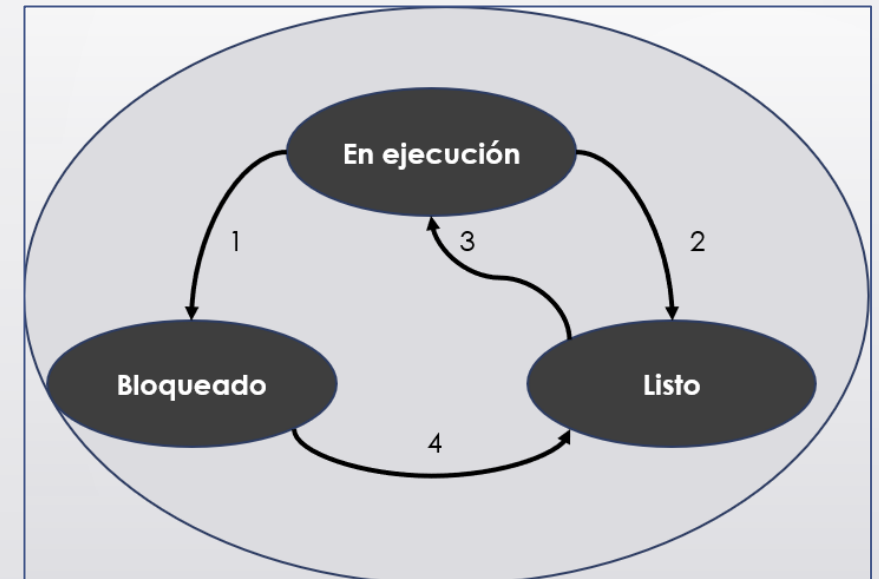
Introducción

- ¿Cuántos procesos se están ejecutando en tu computadora?
`ps -e | wc -l` ó `ps -e`
- ¿Cómo es posible que todos esos procesos estén ejecutándose en el CPU?
- ¿Quién entra primero?
- ¿Qué ocurre con los demás?
- ¿Qué ocurre con los procesos con tiempos largos y con tiempos cortos?



¿Por qué es necesaria la planificación?

- Hay varios procesos en estado “**Listo**”.
- Sólo hay una CPU disponible, hay que decidir cuál proceso se va a ejecutar a continuación.
- ¿Quién decide qué proceso obtiene el CPU y por cuánto tiempo?
- La parte del sistema operativo que realiza esa decisión se conoce como **planificador de procesos (scheduler)**.



Planificador de procesos

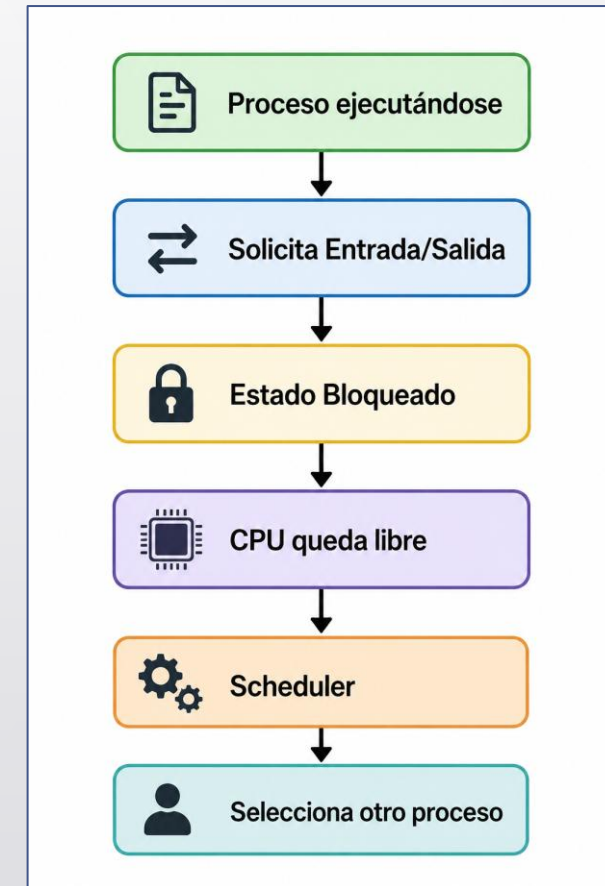
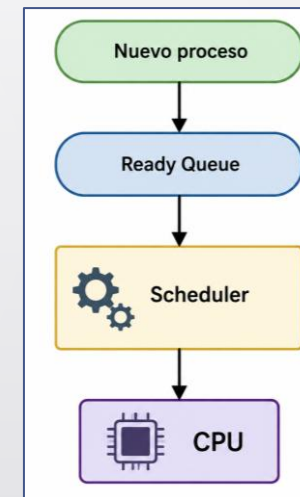
Es un componente del **kernel** del sistema operativo, realiza lo siguiente:

- ✓ Mantiene la lista de procesos listos (**Ready Queue**).
- ✓ Selecciona el siguiente proceso.
- ✓ **Aplica un algoritmo de planificación.**
- ✓ Solicita el cambio de proceso al Dispatcher.
- ✓ Se “preocupa” por utilizar de forma eficiente la CPU, debido a que la **conmutación de procesos es cara**.



¿Cuándo toma las decisiones el Scheduler?

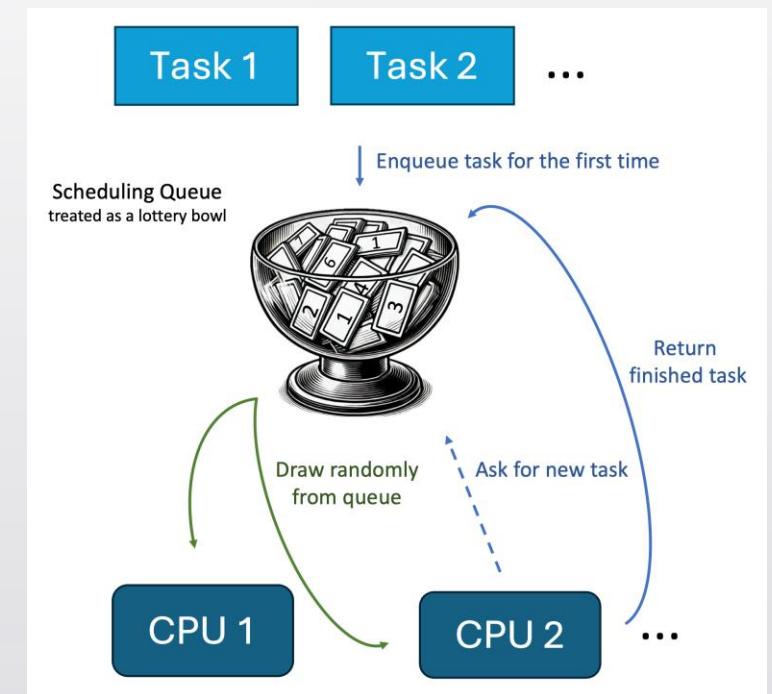
1. Un proceso es creado.
2. Un proceso termina.
3. Un proceso se bloquea esperando un recurso.
4. Finaliza el tiempo asignado al proceso (Quantum).



Objetivos de un Algoritmo de Planificación

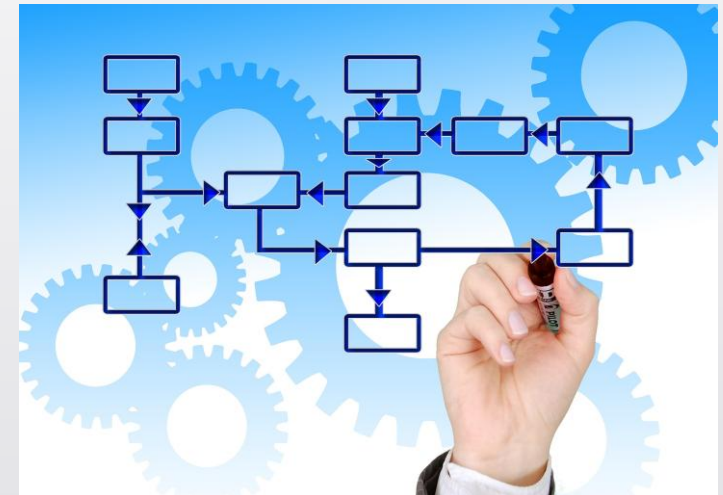
Los algoritmos de planificación buscan:

- **Maximizar la utilización del CPU.** El CPU debe permanecer ocupado la mayor parte del tiempo. Si el CPU está inactivo cuando existen procesos listos se desperdician recursos.
- **Maximizar el rendimiento (Throughput).** Es el número de procesos que puede terminar el sistema por unidad de tiempo.
- **Minimizar el tiempo de retorno.** Es el tiempo total que transcurre desde que un proceso llega al sistema hasta que termina (espera, ejecución y posibles bloqueos).



Objetivos de un Algoritmo de Planificación

- **Minimizar el tiempo de espera.** Es el tiempo durante el cual el proceso permanece esperando en la cola de espera.
- **Minimizar el tiempo de respuesta.** Es el primer instante en el que el usuario obtiene una respuesta después de ejecutar un proceso.
- **Ser justo con todos los procesos (Fairness).** Todos los procesos deben tener oportunidad de utilizar el procesador. Puede ocurrir inanición

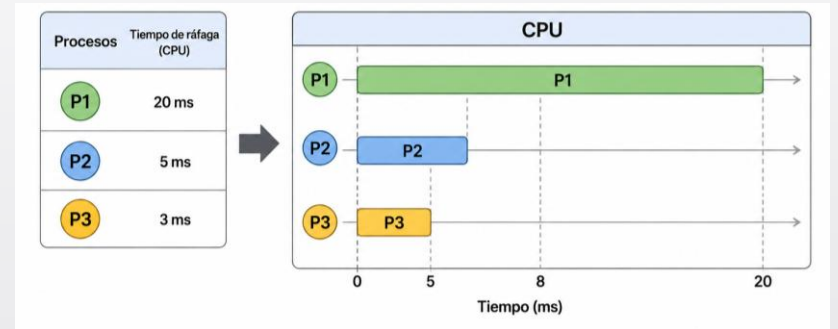


Clasificación de los Algoritmos de Planificación

Los algoritmos de planificación pueden clasificarse en dos grupos:

1. **No apropiativos (Non-Preemptive).** El proceso conserva el CPU hasta terminar o bloquearse, no existe interrupción, pocos cambios de contexto, Implementación sencilla.

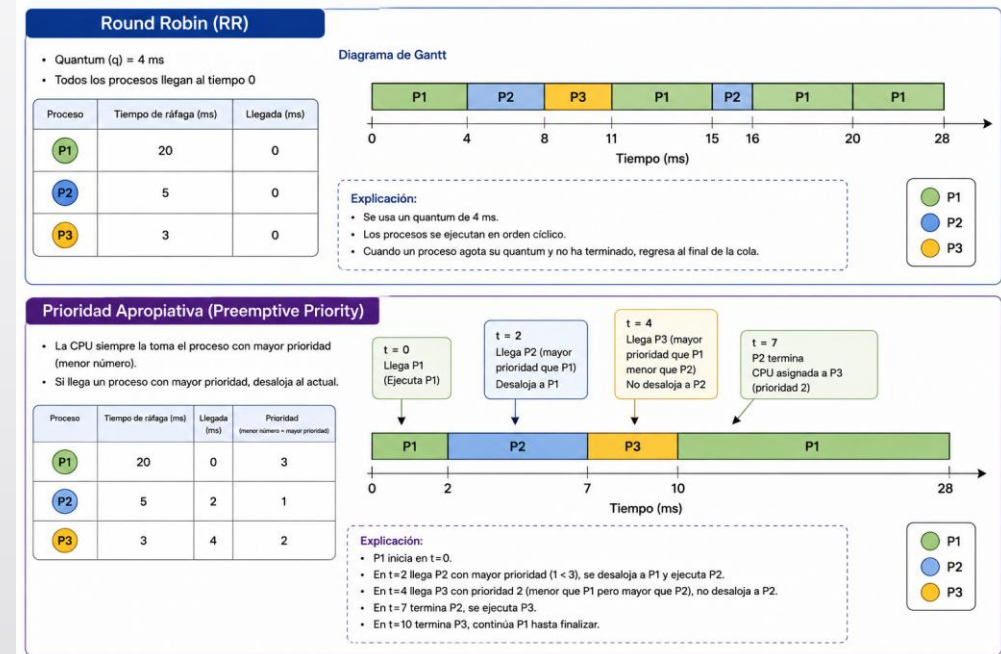
Los algoritmos no apropiativos son: *First Come First Served (FCFS)* y *Shortest Job First (SJF)*



Clasificación de los Algoritmos de Planificación

2. **Apropiativos (Preemptive).** El sistema operativo puede retirar del CPU a un proceso antes de que termine, mayor número de cambios de contexto, Implementación más compleja.

Los algoritmos apropiativos son: Round Robin (RR), Prioridad y múltiples colas.

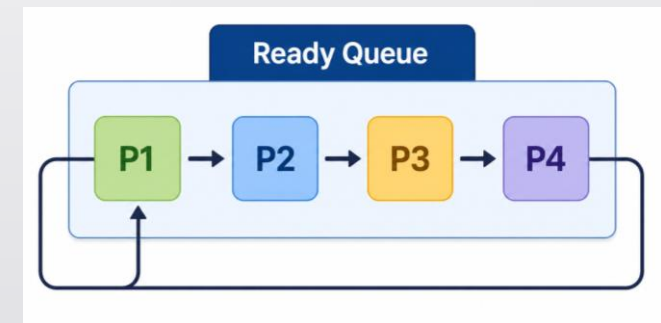


Algoritmo de Planificación Round Robin (RR)

Es un algoritmo de planificación apropiativo que asigna a cada proceso un intervalo fijo de tiempo llamado **Quantum**.

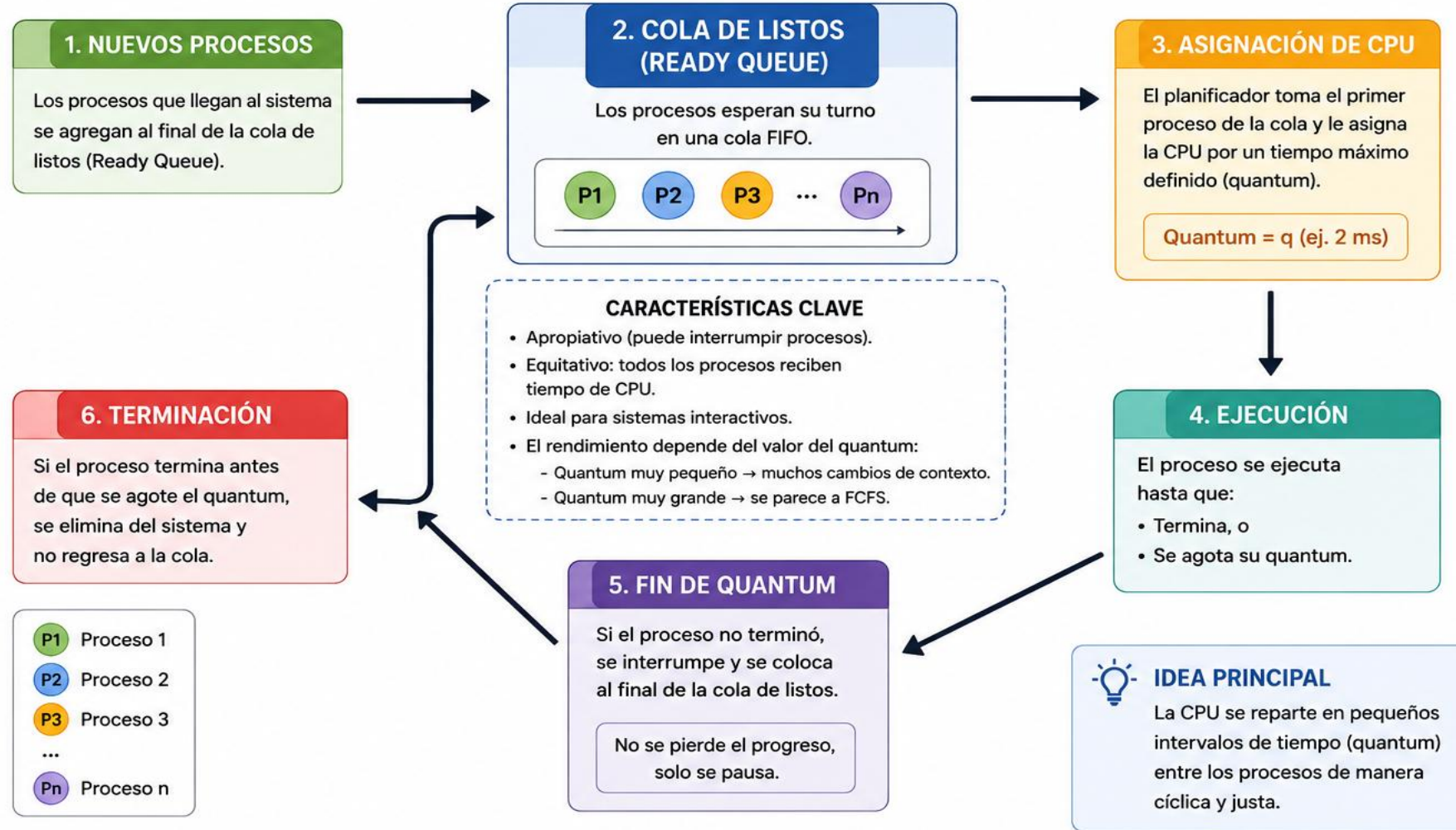
- Cuando un proceso consume su **quantum**:
 - ✓ **Si** terminó su ejecución → Sale del sistema.
 - ✓ **Si no** terminó → Es interrumpido y colocado al final de la cola de procesos listos.

Todos los procesos reciben la misma oportunidad de utilizar el CPU.

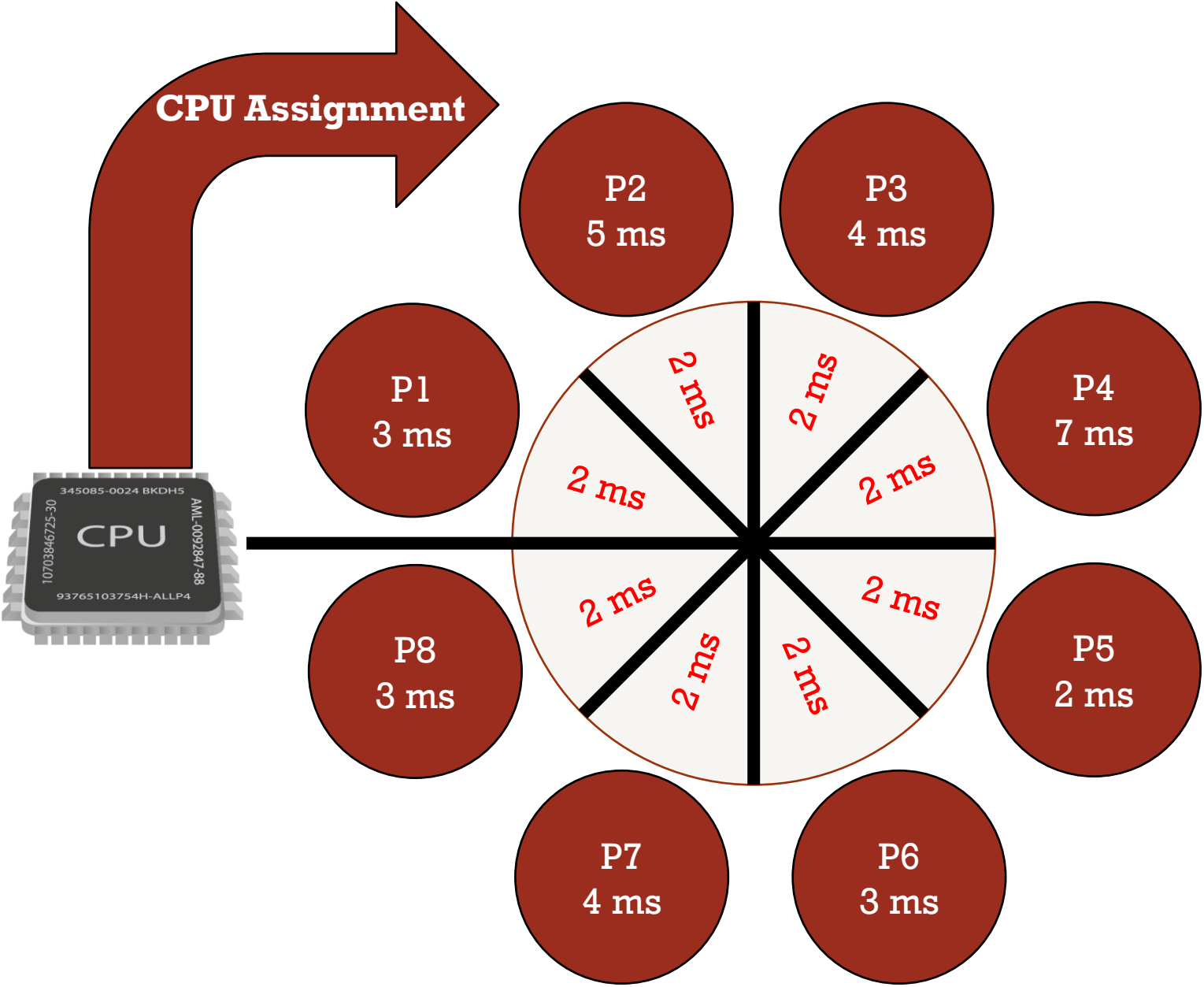


ROUND ROBIN (RR)

Algoritmo de planificación apropiativo basado en tiempo compartido.



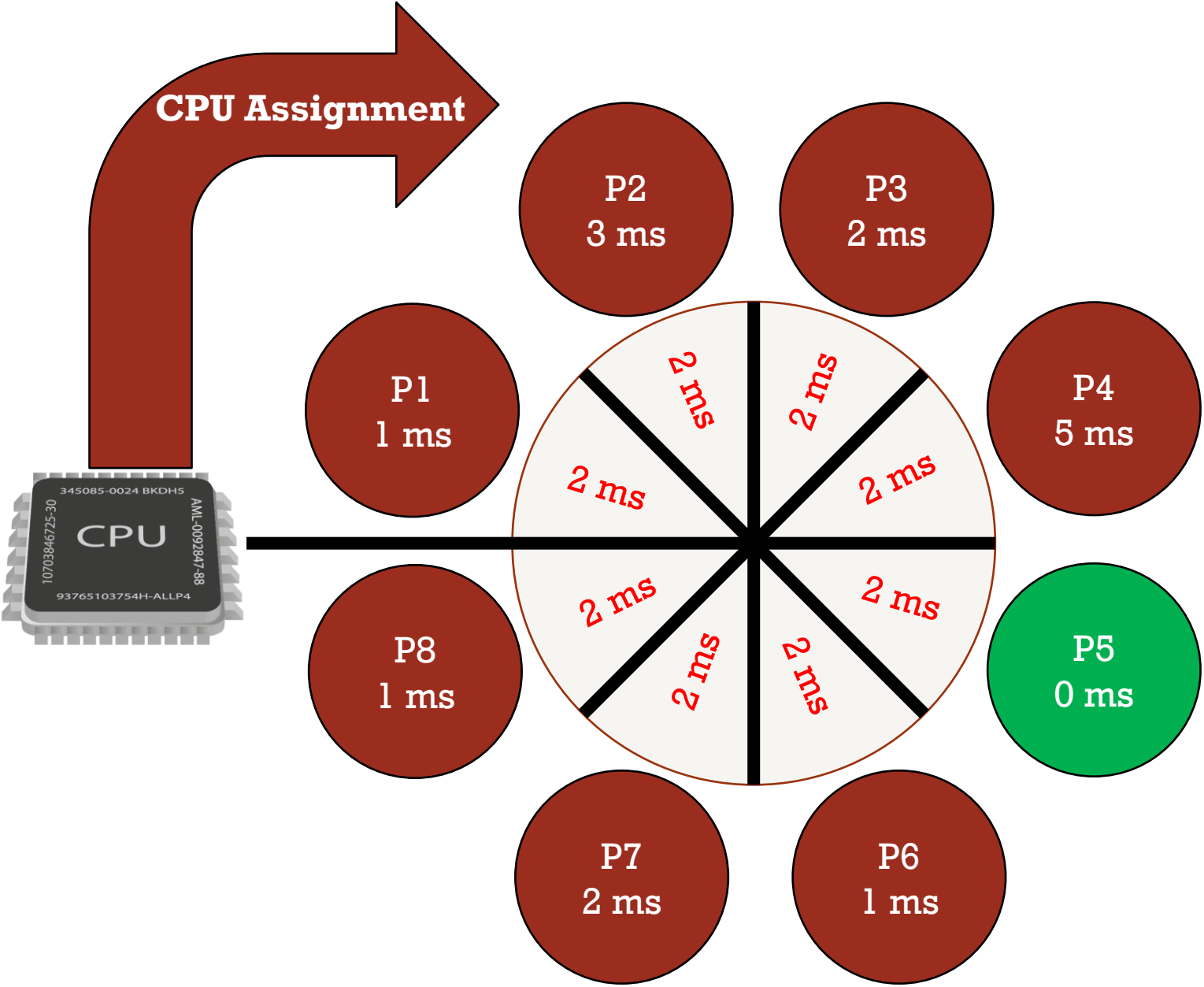
- **Planificación por turno circular.**
Quantum -> Intervalo de tiempo



Primera ronda



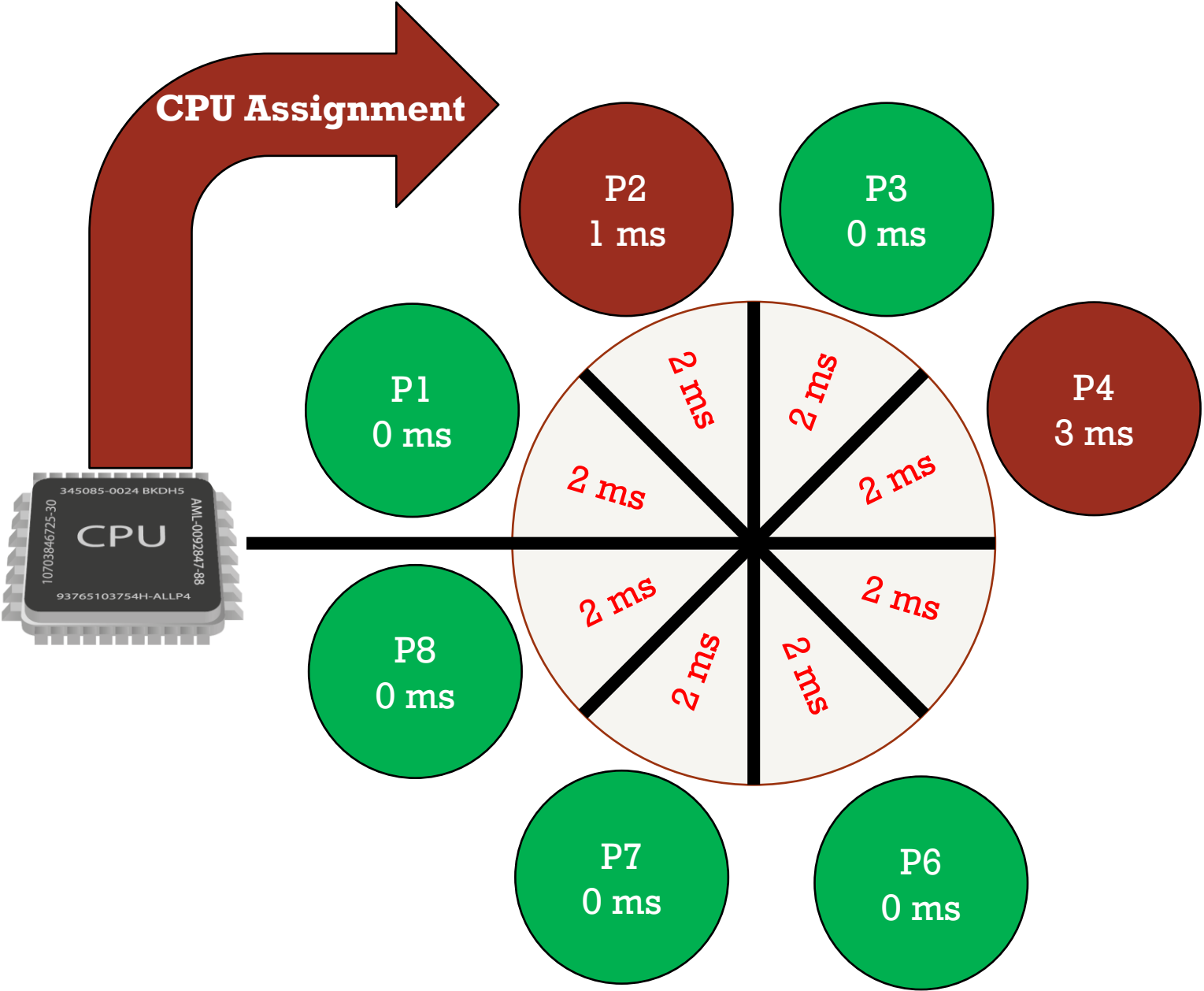
- **Planificación por turno circular.**
Quantum -> Intervalo de tiempo



Segunda ronda



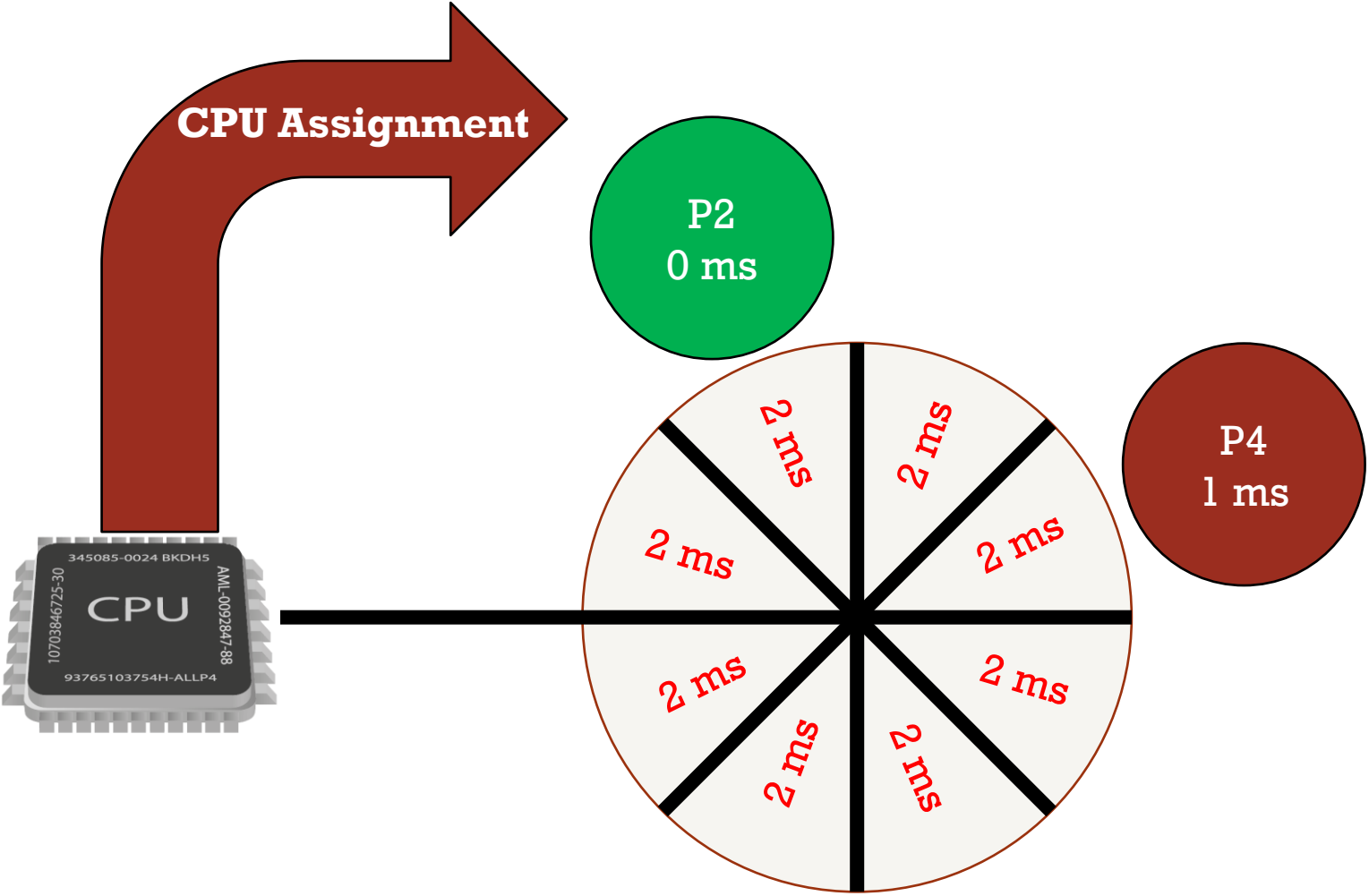
- **Planificación por turno circular.**
Quantum -> Intervalo de tiempo



Tercera ronda



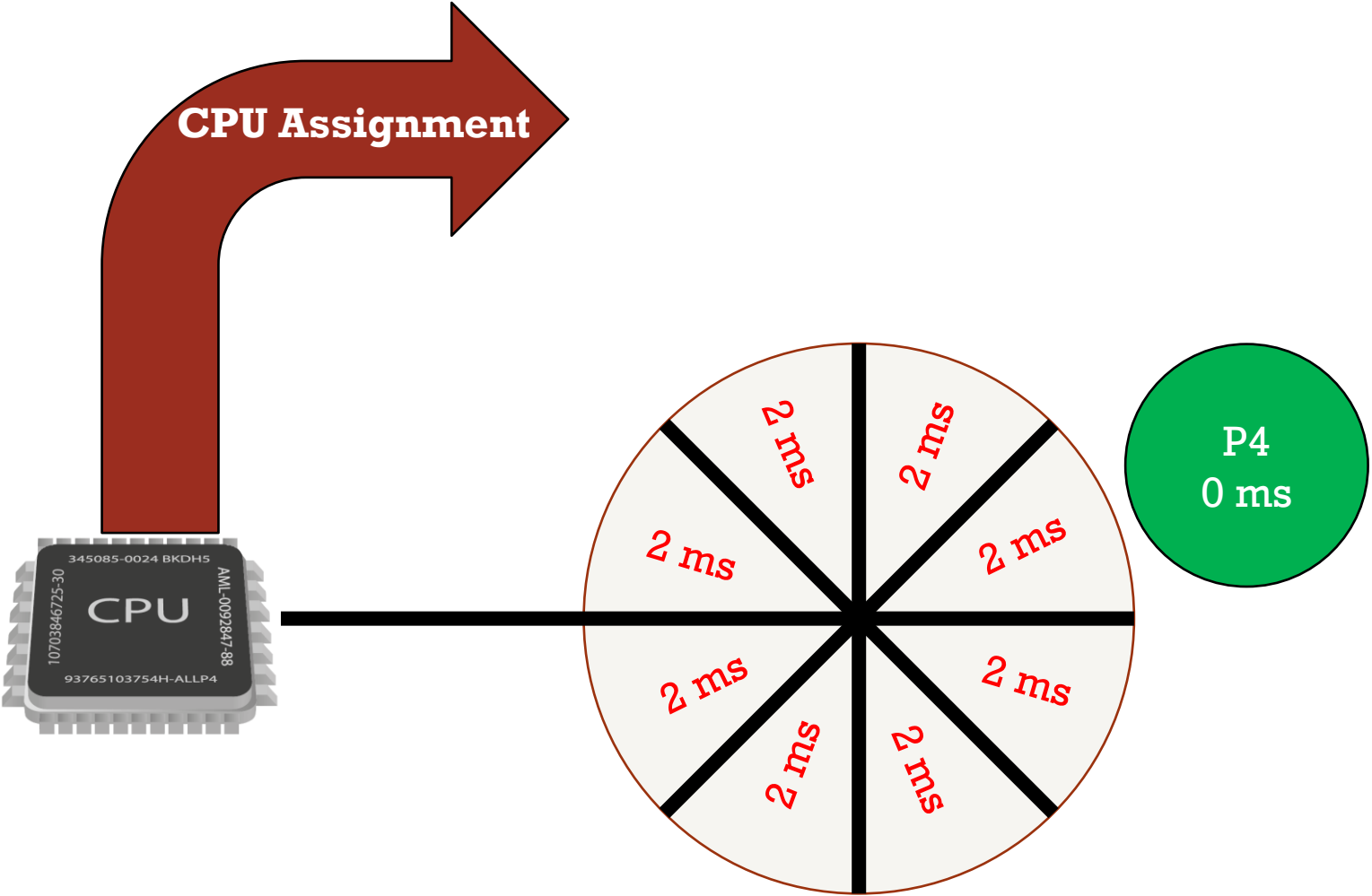
- **Planificación por turno circular.**
Quantum -> Intervalo de tiempo



Cuarta ronda



- **Planificación por turno circular.**
Quantum -> Intervalo de tiempo



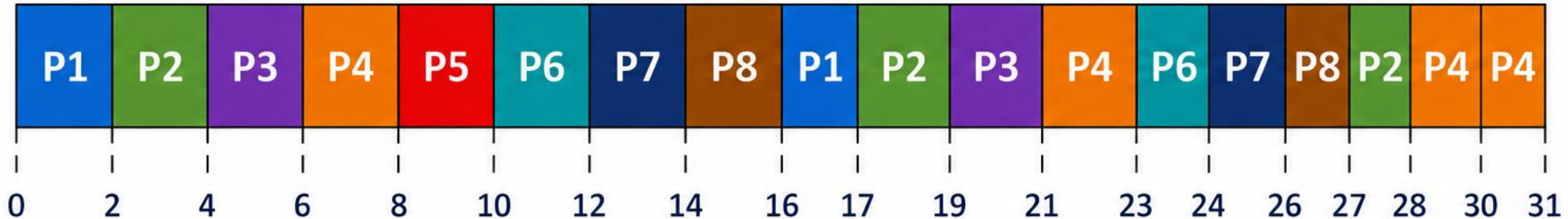
Tercera ronda



ROUND ROBIN

Quantum = 2 ms

DIAGRAMA DE GANTT



ROUND ROBIN – DIAGRAMA DE GANTT (RESULTADO FINAL)

Quantum = 2 unidades de tiempo

Datos del ejemplo

Proceso	Ráfaga inicial (unidades de tiempo)
P1	3
P2	5
P3	4
P4	7
P5	2
P6	3
P7	4
P8	3

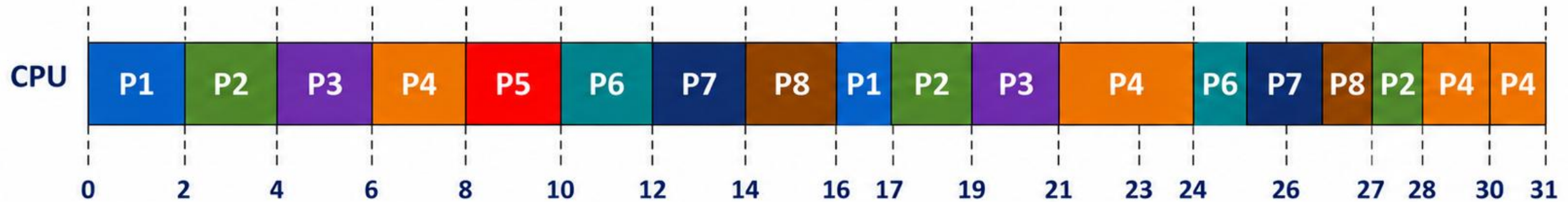
- 8 procesos llegan en el tiempo 0
- Quantum = 2 unidades de tiempo
- El CPU se asigna en orden circular
- Si un proceso termina antes del quantum, libera el CPU inmediatamente

Leyenda de procesos

P1 = Proceso 1
P2 = Proceso 2
P3 = Proceso 3
P4 = Proceso 4
P5 = Proceso 5
P6 = Proceso 6
P7 = Proceso 7
P8 = Proceso 8

Orden de ejecución (secuencia)

P1 → P2 → P3 → P4 → P5 → P6 → P7 → P8 →
P1 → P2 → P3 → P4 → P6 → P7 → P8 → P2 → P4 → P4

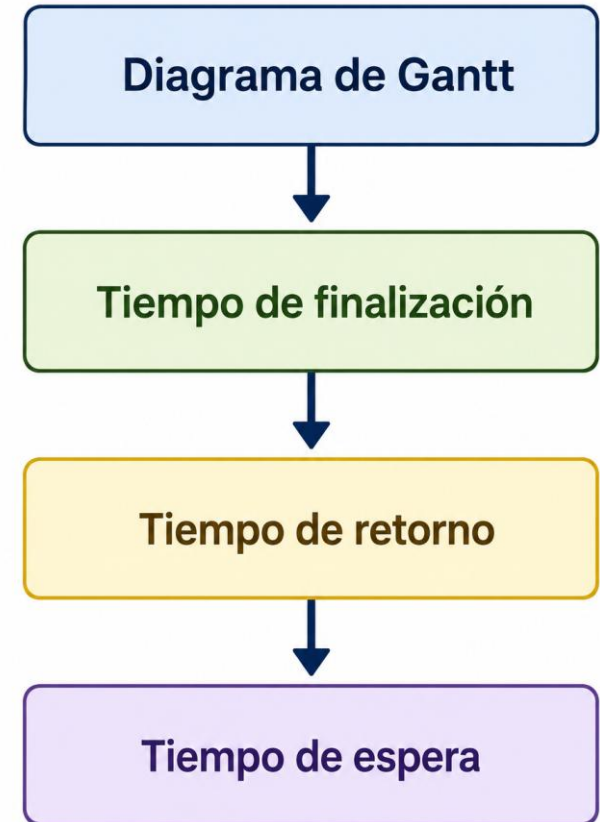


Tiempo total de ejecución = **31** unidades de tiempo.

Métricas de evaluación RR

Después de construir el diagrama de Gantt se puede calcular:

- **Tiempo de finalización.** El instante en el que un proceso termina completamente su ejecución.
- **Tiempo de retorno.** Cuánto tiempo permaneció un proceso dentro del sistema
- **Tiempo de espera.** Cuánto tiempo pasó esperando turno para utilizar el CPU.





Ejemplo 1

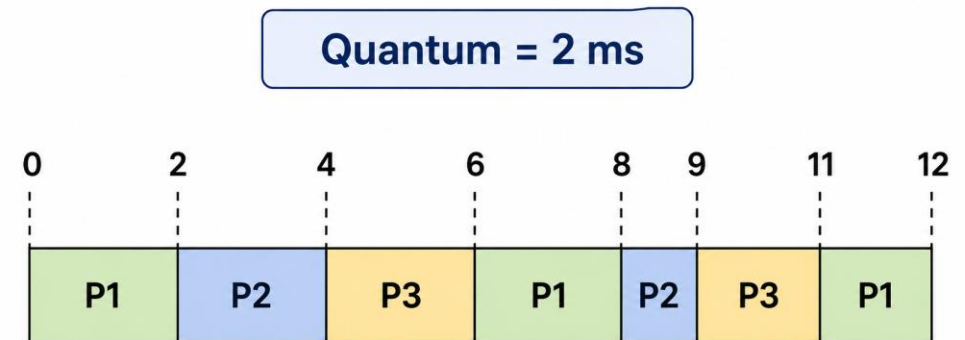
- Toma en cuenta los siguientes procesos.
- Crea el diagrama de Gantt

Proceso	Tiempo de llegada	Ráfaga
P1	0	5
P2	0	3
P3	0	4

Ejemplo 1

- Toma en cuenta los siguientes procesos.
- Crea el diagrama de Gantt

Proceso	Tiempo de llegada	Ráfaga
P1	0	5
P2	0	3
P3	0	4

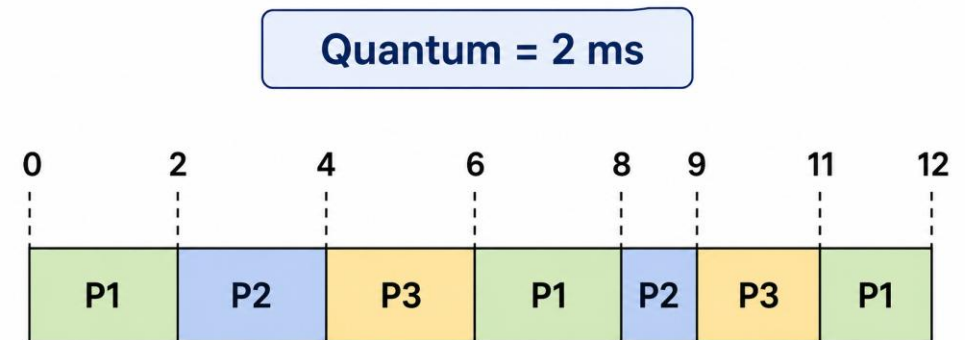


Métricas

- Tiempo de Finalización

Proceso	Finalización
P1	12
P2	9
P3	11

Proceso	Tiempo de llegada	Ráfaga
P1	0	5
P2	0	3
P3	0	4



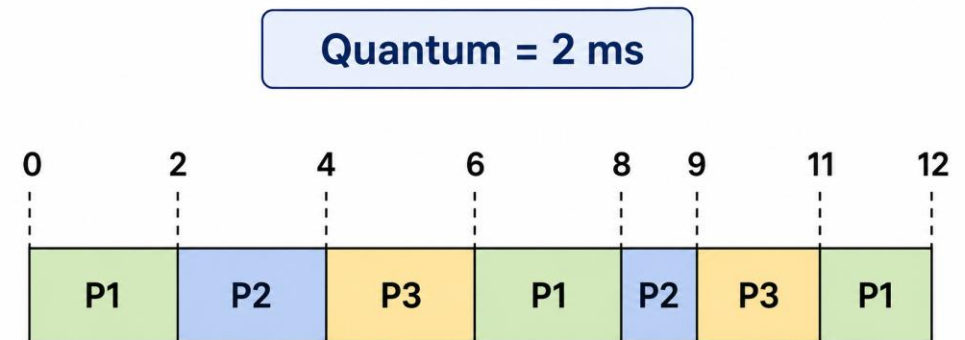
Métricas

- Tiempo de Retorno (Turnaround Time)

$$T_{\text{Retorno}} = T_{\text{Finalización}} - T_{\text{Llegada}}$$

Proceso	T Retorno
P1	12
P2	9
P3	11

Proceso	Tiempo de llegada	Ráfaga
P1	0	5
P2	0	3
P3	0	4



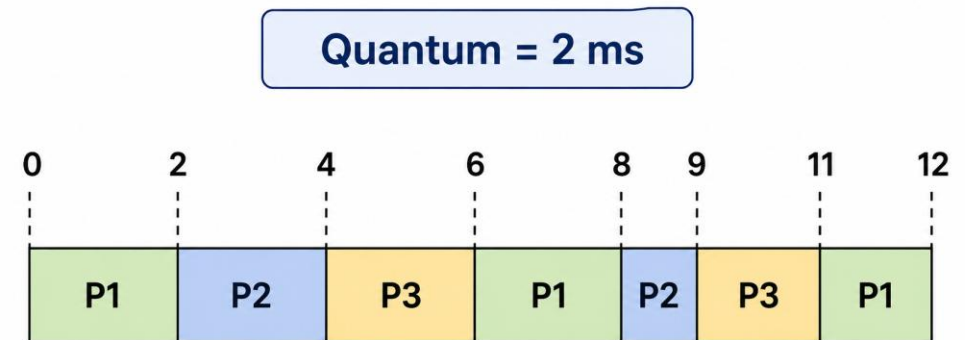
Métricas

- Tiempo de Espera (Waiting Time)

$$T_{Espera} = T_{Retorno} - T_{Rafaga}$$

Proceso	Espera
P1	7
P2	6
P3	7

Proceso	Tiempo de llegada	Ráfaga
P1	0	5
P2	0	3
P3	0	4



Métricas

- Finalmente se obtiene el tiempo de respuesta o retorno **promedio** y el tiempo de espera **promedio**:

Proceso	T Retorno
P1	12
P2	9
P3	11

Proceso	Espera
P1	7
P2	6
P3	7

Fórmula

$$T_{\text{Retorno}} = \frac{\sum T_{\text{Retorno}_i}}{n}$$

Ejemplo

$$\frac{12 + 9 + 11}{3} = 10.66 \text{ ms}$$

Fórmula

$$T_{\text{Espera}} = \frac{\sum T_{\text{Espera}_i}}{n}$$

Ejemplo

$$\frac{7 + 6 + 7}{3} = 6.67 \text{ ms}$$

Ejemplo 2

Q=2

- Realiza el siguiente ejercicio: Obtén el diagrama de Gantt, calcula las métricas de finalización, retorno y de espera. Además de los promedios de las dos últimas y compáralas con el ejercicio anterior.

¿Tiene algo que ver que los procesos hayan cambiado la ráfaga?

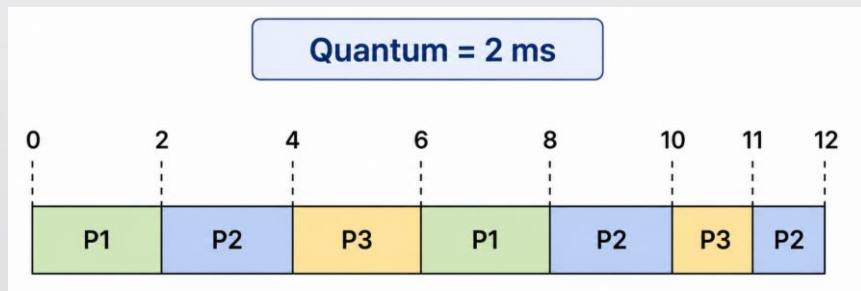
Proceso	Tiempo de llegada	Ráfaga
P1	0	4
P2	0	5
P3	0	3

Resultados

Proceso	Tiempo de llegada	Ráfaga
P1	0	4
P2	0	5
P3	0	3

Proceso	Ráfaga	Llegada	Finalización	Retorno	Espera
P1	4	0	8	8	4
P2	5	0	12	12	7
P3	3	0	11	11	8

Tiempo de retorno promedio: 10.33 ms
Tiempo de espera promedio: 6.33 ms



Ejemplo 3

- Realiza el siguiente ejercicio con las métricas y diagrama de Gantt. Ten en cuenta que no todos los procesos llegaron en el instante 0.

Q=2

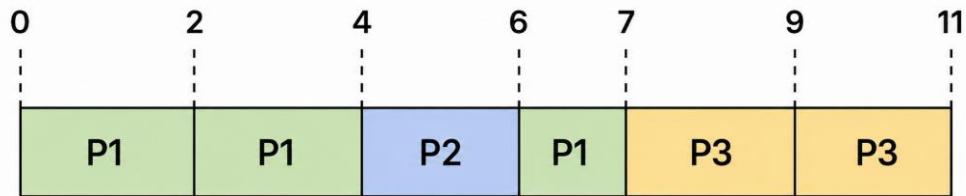
Proceso	Tiempo de llegada	Ráfaga
P1	0	5
P2	4	2
P3	5	4

Resultados

Proceso	Tiempo de llegada	Ráfaga
P1	0	5
P2	4	2
P3	5	4

Proceso	Llegada	Ráfaga	Finalización	Retorno	Espera
P1	0	5	7	7	2
P2	4	2	6	2	0
P3	5	4	11	6	2

Quantum = 2 ms



Tiempo de retorno promedio: 5 ms
Tiempo de espera promedio: 1.33 ms

Algoritmo Round Robin

Ventajas

- ✓ Es un algoritmo justo (Fairness).
- ✓ Todos los procesos reciben tiempo de CPU.
- ✓ Evita la inanición (Starvation).

Desventajas

- ✗ Un Quantum muy pequeño provoca muchos cambios de contexto.
- ✗ Un Quantum muy grande hace que se comporte como FCFS.
- ✗ No considera la prioridad de los procesos.

Mientras existan procesos listos Hacer

P ← Primer proceso de la cola Ready

Ejecutar P durante un Quantum (o hasta que P se bloquee/termine)

Si P terminó Entonces

Eliminar P de la cola y liberar recursos

Sino Si P se bloqueó por Entrada/Salida Entonces

Mover P a la cola de Espera (Waiting Queue)

Sino

// Expropiación de Quantum

Insertar P al final de la cola Ready

Fin Si

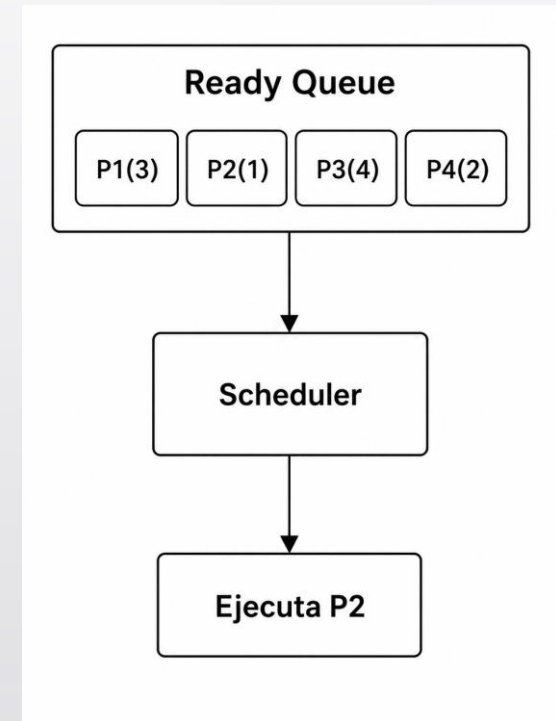
Fin Mientras

Fin

Algoritmo de Planificación por prioridad

Es un algoritmo que puede ser **apropiativo** o **no apropiativo**, surge de la idea de que no todos los procesos son iguales, unos son más importantes que otros.

1. Selecciona para ejecutar el proceso con la prioridad más alta de la cola de procesos.
2. La prioridad es un entero que representa la importancia de un proceso.
3. Si dos procesos tienen la misma prioridad, se utiliza **FCFS** para decidir cuál ejecuta primero.
4. Se evita la inanición con el envejecimiento.



Algoritmo de Planificación por prioridad

Consideraciones:

- Mientras menor sea el número, mayor será la prioridad.
- No importa que proceso llegó primero.
- No importa cuánto tiempo necesita de ráfaga.
- Lo único que importa es la prioridad al menos que dos o más procesos tengan la misma prioridad.

Proceso	Prioridad
P1	4
P2	2
P3	1
P4	5
P5	3

- ✓ ¿Quién ejecuta primero?
- ✓ ¿Quién ejecuta después?
- ✓ ¿Quién queda al final?

Algoritmo de Planificación por prioridad

Consideraciones:

- Mientras menor sea el número, mayor será la prioridad.
- No importa que proceso llegó primero.
- No importa cuánto tiempo necesita de ráfaga.
- Lo único que importa es la prioridad al menos que dos o más procesos tengan la misma prioridad.

Proceso	Prioridad
P1	4
P2	2
P3	1
P4	5
P5	3

- ✓ ¿Quién ejecuta primero?
- ✓ ¿Quién ejecuta después?
- ✓ ¿Quién queda al final?

P3 -> P2 -> P5 -> P1 -> P4

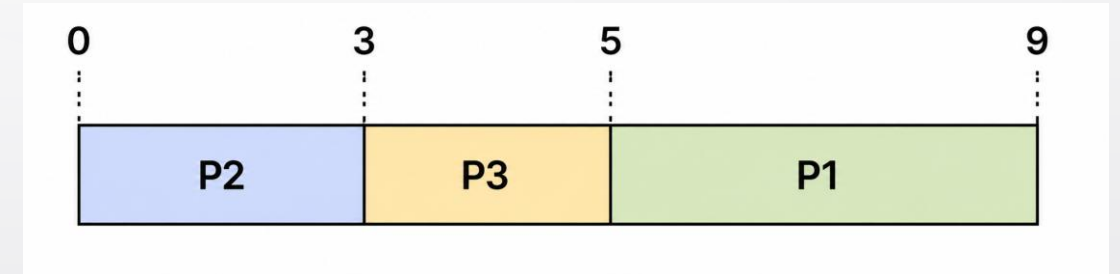
Ejemplo 4

- Crea los diagramas de Gantt utilizando prioridad no apropiativa y apropiativa.

Proceso	Tiempo de llegada	Ráfaga	Prioridad
P1	0	4	3
P2	0	3	1
P3	0	2	2

Ejemplo 4

- Si todos los procesos llegan al mismo tiempo, la planificación por prioridad apropiativa y no apropiativa producen exactamente el mismo diagrama de Gantt.



Proceso	Llegada	Ráfaga	Prioridad	Finalización	Retorno	Espera
P1	0	4	3	9	9	5
P2	0	3	2	3	3	0
P3	0	2	1	5	5	3

Tiempo de retorno promedio: 5.67 ms

Tiempo de espera promedio: 2.67 ms

Ejemplo 5

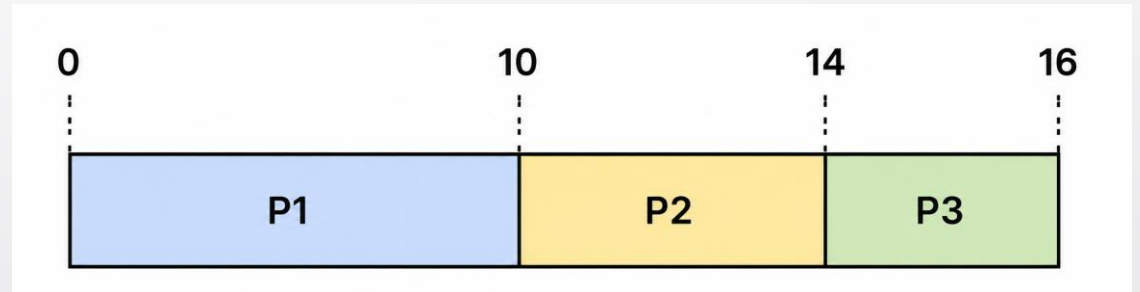
- Crea los diagramas de Gantt utilizando prioridad no apropiativa y apropiativa y sus respectivas métricas.

Proceso	Tiempo de llegada	Ráfaga	Prioridad
P1	0	10	3
P2	2	4	1
P3	4	2	2

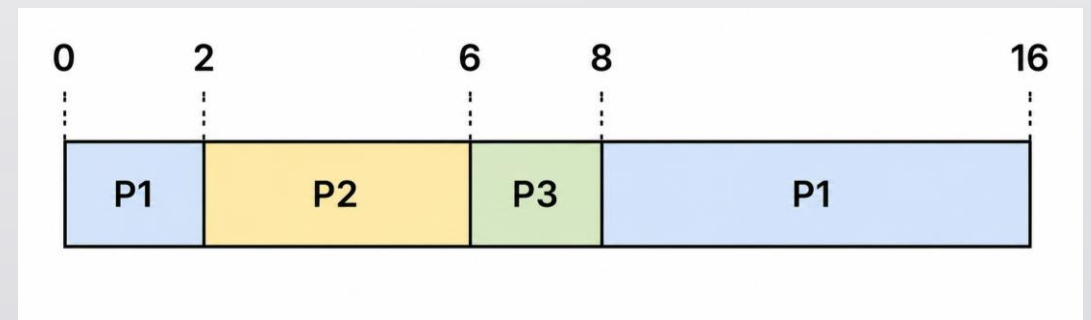
Ejemplo 5

Proceso	Tiempo de llegada	Ráfaga	Prioridad
P1	0	10	3
P2	2	4	1
P3	4	2	2

No apropiativa



Apropiativa



Ejemplo 5

Proceso	Llegada	Ráfaga	Prioridad	Finalización	Retorno	Espera
P1	0	10	3	10	10	0
P2	2	4	1	14	12	8
P3	4	2	2	16	12	10

Tiempo de retorno promedio: 11.33 ms

Tiempo de espera promedio: 6 ms

Proceso	Llegada	Ráfaga	Prioridad	Finalización	Retorno	Espera
P1	0	10	3	16	16	6
P2	2	4	1	6	4	0
P3	4	2	2	8	4	2

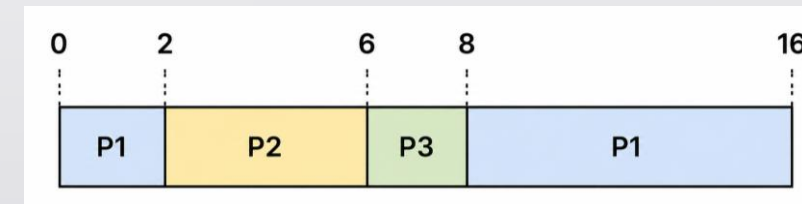
Tiempo de retorno promedio: 8 ms

Tiempo de espera promedio: 2.66 ms

No apropiativa



Apropiativa



Ejemplo 6

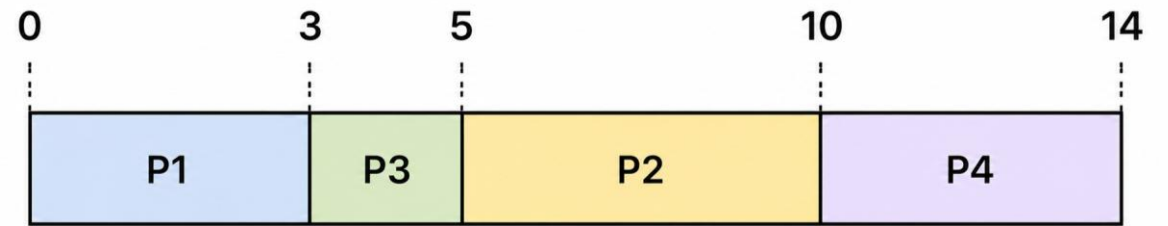
- Crea los diagramas de Gantt utilizando prioridad no apropiativa y apropiativa y sus respectivas métricas.

Proceso	Tiempo de llegada	Ráfaga	Prioridad
P1	0	3	4
P2	1	5	2
P3	1	2	1
P4	4	4	3

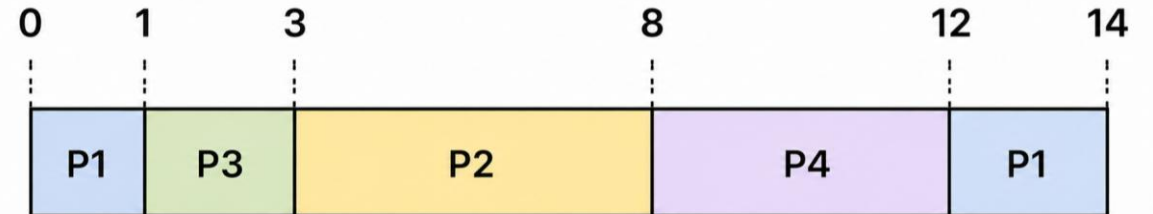
Ejemplo 6

Proceso	Tiempo de llegada	Ráfaga	Prioridad
P1	0	3	4
P2	1	5	2
P3	1	2	1
P4	4	4	3

No apropiativa



Apropiativa



Ejemplo 6

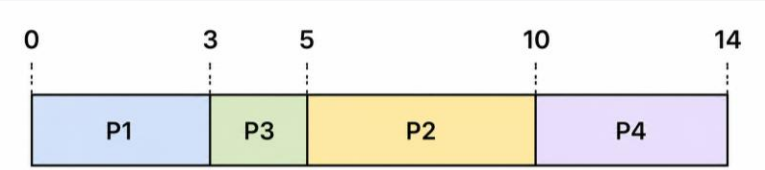
Proceso	Llegada	Ráfaga	Prioridad	Finalización	Retorno	Espera
P1	0	3	4	3	3	0
P2	1	5	2	10	9	4
P3	1	2	1	5	4	2
P4	4	4	3	14	10	6

Tiempo de retorno promedio: 6.5 ms
Tiempo de espera promedio: 3 ms

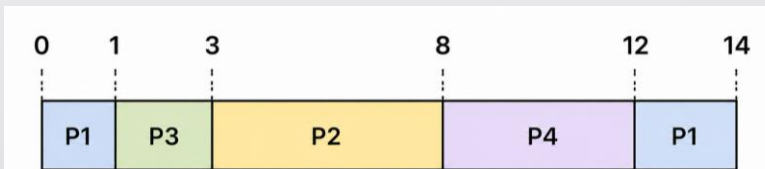
Proceso	Llegada	Ráfaga	Prioridad	Finalización	Retorno	Espera
P1	0	3	4	14	14	11
P2	1	5	2	8	7	2
P3	1	2	1	3	2	0
p4	4	4	3	12	8	4

Tiempo de retorno promedio: 7.75 ms
Tiempo de espera promedio: 4.25 ms

No apropiativa



Apropiativa



Algoritmo por prioridad



No apropiativo

Mientras existan procesos por ejecutar hacer

Buscar en la cola Ready el proceso con mayor prioridad

Asignar el CPU al proceso seleccionado

Ejecutar el proceso hasta que termine

Retirar el proceso del sistema

Fin Mientras

Apropiativo

Mientras existan procesos por ejecutar hacer

Si llega un proceso con mayor prioridad entonces

Interrumpir el proceso actual

Guardar su contexto

Asignar el CPU al nuevo proceso

Fin Si

Fin Mientras

Algoritmo por prioridad

Ventajas

- ✓ Ejecuta primero los procesos más importantes.
- ✓ Muy adecuado para sistemas de tiempo real.
- ✓ Reduce el tiempo de respuesta de procesos críticos.
- ✓ Es flexible, ya que las prioridades pueden modificarse.
- ✓ Permite distinguir entre procesos del sistema y procesos de usuario.

Desventajas

- ✗ Los procesos con baja prioridad pueden esperar demasiado tiempo.
- ✗ Puede presentarse **inanición (Starvation)**.
- ✗ Es necesario asignar correctamente las prioridades.
- ✗ La implementación es más compleja que FCFS o Round Robin.
- ✗ Si las prioridades nunca cambian, algunos procesos podrían no ejecutarse.



Tarea

Implemente en lenguaje **C** los algoritmos de planificación **Round Robin** y **Prioridad Apropiativa**.

La simulación puede realizarse de la forma que considere más conveniente; sin embargo, el programa deberá representar correctamente el funcionamiento de cada algoritmo.

Además, para el mismo conjunto de procesos utilizado en la simulación, entregue en papel lo siguiente:

1. El **diagrama de Gantt** generado por cada algoritmo.
2. La tabla de métricas de cada proceso:
 - Tiempo de finalización.
 - Tiempo de retorno.
 - Tiempo de espera.
 - El tiempo de retorno promedio.
 - El tiempo de espera promedio.

Durante la revisión deberá explicar el funcionamiento de su código y realizar las modificaciones que se le soliciten, por lo que es indispensable comprender completamente su implementación.

Ejemplo 7



- Construye los diagramas de Gantt utilizando los algoritmos de planificación por **prioridad no apropiativa** y **prioridad apropiativa**, calculando en cada caso las métricas correspondientes. Durante la resolución del ejercicio deberás aplicar la técnica de **envejecimiento (aging)**, cuyo objetivo es evitar que procesos con menor prioridad permanezcan indefinidamente al final de la cola debido a la llegada continua de procesos con mayor prioridad. Para este sistema operativo, la regla de envejecimiento será la siguiente:
- Por cada **2 ms** que un proceso permanezca esperando en la cola de listos, su prioridad aumentará automáticamente un nivel. Es decir, el valor numérico de su prioridad disminuirá en una unidad (por ejemplo, de prioridad **4** a **3**).
- Considera que la prioridad más alta es **0**; por lo tanto, ningún proceso podrá tener prioridades negativas. En la versión **apropiativa**, cada vez que el mecanismo de **aging** modifica la prioridad de un proceso en espera, el Scheduler vuelve a evaluar la cola de listos. Si existe un proceso con prioridad estrictamente mayor que la del proceso en ejecución, éste es interrumpido.

Proceso	Tiempo de Llegada	Ráfaga	Prioridad
P1	0	6	4
P2	0	2	1
P3	1	3	1
P4	3	4	1